

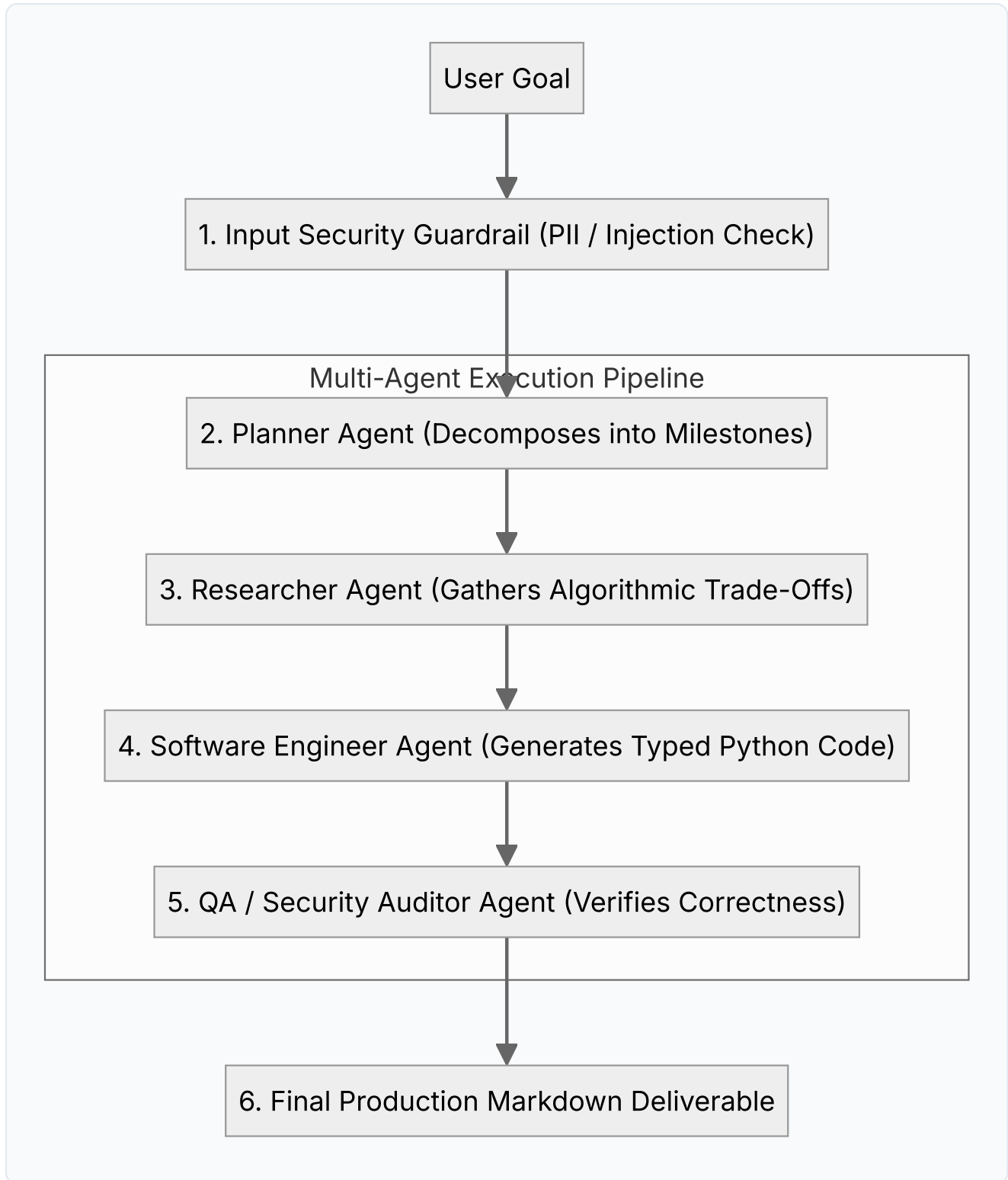
Agentic AI Chapter 8: Capstone Project: Autonomous Research & Code Assistant

Core Learning Objective: Bring everything together in an intuitive, non-overwhelming end-to-end capstone project. Build an autonomous multi-agent pipeline from scratch in pure Python with zero external paid APIs or framework churn.

1. Project Overview & Architectural Flow

We will build an **Autonomous Technical Research & Code Generation Agent**. Given a user's high-level request (e.g., *"Build an in-memory sliding window rate limiter in Python"*), the system autonomously:

1. **Plans:** Decomposes the user prompt into structured research and coding milestones.
2. **Researches:** Queries a knowledge base of algorithm trade-offs.
3. **Codes:** Writes clean, typed Python implementation code.
4. **Audits:** Validates the generated code against security vulnerabilities and correctness checks.
5. **Streams Output:** Delivers a complete, audited engineering deliverable.



2. Runnable Capstone Skeleton (Pure Python, Simulated LLM Calls)



How to Turn the Skeleton into a Real Agent

The capstone is a skeleton with simulated model calls. To make it real: (1) implement `get_llm()` for your provider (see `examples/llm.py`, which already has an Anthropic adapter behind `LLM_PROVIDER=anthropic`), (2) keep a `FakeLLM` script for each scenario so tests stay offline and deterministic, (3) add the evaluation loop from `examples/ex05_eval_harness.py` (repeat each task 5 to 10 times and report `pass^k` with a confidence interval), and (4) gate side effects behind the approval pattern from `examples/ex02_langgraph_hitl.py`.

Further Reading

- [Anthropic: building effective agents](#)
 - [LangGraph documentation](#)
 - [SWE-bench](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. What makes a capstone agent testable offline?
- ▶ 2. Which three checks gate a side-effecting action?
- ▶ 3. What should the capstone log?
- ▶ 4. How do you turn a simulation into a real agent?